1) Simulated annealing

```
import random
import math


def calculate_cost(board):
    """Count pairs of queens attacking each other."""
    n = len(board)
    cost = 0

    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                cost += 1

    return cost


def simulated_annealing(n=8):
    max_restarts = 100

    for restart in range(max_restarts):
        # One queen in each column, at a random row
        current = [random.randrange(n) for _ in range(n)]
        current_cost = calculate_cost(current)

        best = current[:]
        best_cost = current_cost

        temperature = 100.0
        cooling_rate = 0.995
        min_temperature = 0.001

        while temperature > min_temperature:
            if current_cost == 0:
                return current, current_cost

            # Create a neighbor by moving one queen
            neighbor = current[:]
            col = random.randrange(n)

            new_row = random.randrange(n - 1)
```

```
            if new_row >= neighbor[col]:
                new_row += 1

            neighbor[col] = new_row
            neighbor_cost = calculate_cost(neighbor)

            delta = neighbor_cost - current_cost

            # Accept improvements; sometimes accept worse states
            if delta < 0 or random.random() < math.exp(
                -delta / temperature
            ):
                current = neighbor
                current_cost = neighbor_cost

            if current_cost < best_cost:
                best = current[:]
                best_cost = current_cost

            temperature *= cooling_rate

        if best_cost == 0:
            return best, best_cost

        print(
            "Restart", restart + 1,
            "- best cost:", best_cost
        )

    return best, best_cost


def display_board(board):
    n = len(board)

    for row in range(n):
        for col in range(n):
            if board[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
```

```
solution, cost = simulated_annealing(8)

print("\nFinal board:", solution)
print("Final cost:", cost)

if cost == 0:
    print("Solution found!")
else:
    print("No solution found. Run the program again.")

display_board(solution)
```

File Edit Shell Debug Options Window Help

```
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr  9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/STUDENT/Desktop/neha/annealing.py

Final board: [6, 1, 5, 2, 0, 3, 7, 4]
Final cost: 0
Solution found!
. . . . Q . . .
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
Q . . . . . . .
. . . . . Q . .
>>>
```